

# **Laporan Tugas Kecil 1: Implementasi Algoritma Brute Force**

Reysha Syafitri MR

NIM: 13524137

17 Februari 2026

# 1 Penjelasan Algoritma Brute Force

Pada bagian ini, jelaskan algoritma yang Anda gunakan secara naratif. Jangan hanya menuliskan pseudocode, tetapi jelaskan alur logika dan langkah-langkahnya.

## 2 pseudocode

Listing 1: notasi algoritma

```
1 KAMUS GLOBAL
2     coordinates: <x : integer, y : integer>
3     colors_coordinate : map<char, array of coordinates>
4     queens_position: map<char, coordinates>
5     board: array of string
6
7 procedure ShiftQueens (input/output queens_position: map<char
8     , coordinates>)
9     {I.S setiap queen di queens_position terletak di petak
10     awal masing2 warna
11     F.S jika ada petak yang conflict, maka terjadi
12     penyesuaian petak setiap queen pada queens_position}
13 KAMUS
14     colors : array of char
15     n : integer
16     indices: array of integer
17     maxIndices: array of integer
18     posA, posB : coordinates
19     tempPos : map<char, coordinates>
20     found, allValid : boolean
21     carry: integer
22     iterationCount : integer
23     startTime, endTime : time
24     waktuMs : real
25
26 ALGORITMA
27     iterationCount <- 0
28     startTime <- currentTime()
29
30     {#1=====}
31     {1. buat daftar warna dari colors_coordinate: }
```

```

29   for each pair in queens_position do
30       colors.add(color)
31   n <- colors.length
32
33   {#2=====}
34   {2. inisialisasi index setiap warna: }
35   indices <- array of n elements, all 0
36
37   {#3=====}
38   {3. hitung jumlah koordinat maks setiap warna: }
39   for i <- 0 to n-1 do
40       maxIndices[i] <- colors_coordinate[colors[i]].length
41   found <- false
42
43   {#4=====}
44   {4. algoritma brute force }
45   while(!found) do
46       iterationCount <- iterationCount + 1
47
48       {#4a=====}
49       {4a. buat posisi queen sementara dari indices skrg}
50       for i <- 0 to n-1 do
51           tempPos[colors[i]] <- colors_coordinate[colors[i]
52               ][indices[i]]
53
54       {#4b=====}
55       {4b. live update: tampilkan proses brute force setiap
56           1 detik}
57       clearScreen()
58       output("Iterasi ke-", iterationCount)
59       printBoard(tempPos)
60       sleep(1000) {tunggu 1 detik (1000 ms)}
61
62       {4c. cek apakah terjadi konflik antar queen}
63       allValid <- true
64       for i <- 0 to n-1 do
65           for j <- i+1 to n-1 do
66               posA <- tempPos[colors[i]]
67               posB <- tempPos[colors[j]]

```

```

67         {cek baris, kolom, dan diagonal yg
        bersentuhan}
68     if (posA.x = posB.x) or
69        (posA.y = posB.y) or
70        (abs(posA.x - posB.x) = 1 AND abs(posA.y
        - posB.y) = 1) then
71        allValid <- false
72
73     {#4c=====}
74     {4c. if valid, simpan solusi}
75     if allValid then
76         queens_position <- tempPos
77         found <- true
78     else
79         {4d. increment indices}
80         carry <- n-1
81         while carry >= 0 do
82             indices[carry] <- indices[carry] +1
83             if(indices[carry] < maxIndices[carry]) then
84                 break
85             indices[carry] <- 0
86             carry <- carry -1
87
88         {4e. if semua kombinasi sdh dicoba tp ga ktemu}
89         if (carry < 0) then
90             output("Tidak ada solusi!")
91             break
92
93     endTime <- currentTime()
94     waktuMs <- (endTime - startTime) dalam milidetik
95
96     output("hasil akhir:")
97     printBoard(queens_position)
98
99     output("Jumlah konfigurasi yang ditinjau: ",
        iterationCount)
100
101     output("Waktu eksekusi: ", waktuMs, " ms")
102
103

```

```

104 procedure printBoard(input queens_position : map<char,
      coordinates>)
105     {I.S: queens_posistion berisi posisi queen, board berisi
      papan asli
106     F.S: papan ditampilkan dengan posisi queen ditandai '#'}
107 KAMUS
108     output : array of string
109     pos : coordinates
110     line : string
111     colIdx : integer
112 ALGORITMA
113     output <- board
114
115     for each <color, pos> in queens_pos do
116         if(pos.y >= 0 and pos.y < output.length) then
117             colIdx <- 0
118             for i = 0 to output[pos.y].length - 1 do
119                 if(output[pos.y][i] != ' ' and output[pos.y][
120                     i] != '\t') then
121                     if(colIdx = pos.x)then
122                         output[pos.y][i] = '#'
123                         break
124                     colIdx <- colIdx + 1
125
126     for each line in outputBoard do
127         output(line)
128
129
130 {ALGORITMA UTAMA
      =====}
131 function main() -> integer
132 KAMUS
133     filename : string
134     file : ifstream
135     line : string
136     x, y : integer
137     c : char
138     boardWidth : integer
139     charCount : integer

```

```

140     colIdx : integer
141     isValid : boolean
142     color : char
143     coords: array of coordinates
144
145 ALGORITMA
146     {#1 minta input=====}
147     output("masukkan nama file test case (.txt): ")
148     input(filename)
149
150     if filename tidak berakhiran ".txt" then
151         output("Error: file harus berekstensi .txt!")
152         return
153
154     file <- open(filename)
155     if NOT file is open then
156         output("error: tidak dapat buka file ", filename)
157         return
158
159     {#2 baca file dan validasi=====}
160     y <- 0
161     boardWidth <- -1
162     isValid <- true
163
164     while getline(file, line)do
165         {trim spasi di akhir baris}
166         while line tidak kosong AND (karakter terakhir line =
            ' ' OR karakter terakhir line = '\t') do
167             hapus karakter terakhir dari line
168
169         board.add(line)
170
171         if(board.length = 0) then
172             continue
173
174         {#3 hitung lebar papan (hanya karakter non-spasi)
            =====}
175         charCount <- 0
176         for i <- 0 to line.length - 1 do
177             if line[i] != ' ' AND line[i] != '\t' then

```

```

178         charCount <- charCount + 1
179
180     if boardWidth = -1 then
181         boardWidth <- charCount
182     else
183         if charCount != boardWidth then
184             output("Error: ukuran papan tidak konsisten")
185             isValid <- false
186             return
187
188     {simpan koordinat setiap warna (x = index non-spasi)}
189     colIdx <- 0
190     for x <- 0 to line.length - 1 do
191         c <- line[x]
192         if c != ' ' and c != '\t' then
193             colors_coordinate[c].add({colIdx, y})
194             colIdx <- colIdx + 1
195     y <- y + 1
196     close(filename)
197
198     {#4 cek apakah papan kosong=====}
199     if board.length = 0 then
200         output("Error: file kosong")
201         return
202
203     {#5 cek apakah papan persegi=====}
204     if board.length != boardWidth then
205         output("Error: papan tidak NxN")
206         return
207
208     {#6 cek apakah jumlah warna = jumlah kolom/baris}
209     if colors_coordinate.lenthgh != board.length then
210         output("error: jumlh warna hrs sesuai dgn ukuran
                papan")
211         return
212
213     output("input valid")
214     output("jumlah warna: ", colors_coordinate.length)
215
216     {=====ALGORITMA INTI=====}

```

```

217     {#1 tempatin queen di petak awal setiap region warna}
218     for each <color, coords> in colors_coordinate do
219         if NOT coords.empty then
220             queens_posistion[color] <- coords[0]
221
222     {#2 brute force}
223     ShiftQueens(queens_posistion)

```

### 3 Penjelasan langkah-langkah

program dibuat dengan menggunakan algoritma brute force. dengan meminta masukan dari file txt, lalu masukkan koordinat tiap warna ke dalam map dengan key char dan value koordinat (x,y) lalu tiap tiap queen dalam queen\_position diinisialisasi, tiap warna punya 1 queen, dan queen ditempatkan di petak awal dari koordinat masing2 warna. kemudian queen melalui proses brute force, dicoba kombinasi, jika dilihat posisi secara baris, kolom atau secara diagonal bersentuhan ada sepasang queen, maka akan dicoba semua kemungkinan petak koordinat dari region warna terakhir, jika masih belum ditemukannya solusi yang valid, maka semua kemungkinan petak koordinat dari region warna lainnya yang akan dicoba, kombinasi dilakukan secara menyilang (uji koordinat antar warna diuji secara bersamaan). pengujian tiap-tiap kombinasi silang dilakukan hingga ditemukan solusi yang valid atau seluruh kombinasi habis (tidak ditemukan solusi).

## 4 Source Code

Berikut adalah kode sumber program dalam bahasa C++:

### 4.1 main.cpp

Listing 2: Implementasi program utama

```

1     #include <iostream>
2     #include <fstream>
3     #include <string>
4     #include "queens.h"
5
6     int main(){
7         std::string filename;

```



```

8      std::ifstream file;
9      std::string line;
10     int x, y;
11     char c;
12     int boardWidth;
13     bool isValid;
14     char color;
15     std::vector<coordinates> coords;
16
17     //===== INPUT
18     =====
19
20     std::cout << "Masukkan nama file (.txt): ";
21     std::cin >> filename;
22
23     if(filename.size() < 4 || filename.substr(filename.
24         size() - 4) != ".txt"){
25         std::cerr << "error: file tidak berekstensi .txt!"
26             " << std::endl;
27         return 1;
28     }
29
30     file.open(filename);
31     if(!file.is_open()){
32         std::cerr << "error: tidak dapat membuka file!"
33             << filename << std::endl;
34         return 1;
35     }
36
37     //===== BACA FILE
38     =====
39
40     y = 0;
41     boardWidth = -1;
42     isValid = true;
43
44     while(std::getline(file, line)){
45         //trim trailing whitespace (spasi, tab, \r, \n)
46         while(!line.empty() && (line.back() == ' ' ||
47             line.back() == '\t' || line.back() == '\r' ||

```

```

42         line.back() == '\n')){
43             line.pop_back();
44         }
45
46         //skip baris kosong
47         if(line.empty()){
48             continue;
49         }
50
51         board.push_back(line);
52
53         //hitung lebar papan
54         int charCount = 0;
55         for(int i = 0; i < static_cast<int>(line.size());
56             i++){
57             if(line[i] != ' ' && line[i] != '\t'){
58                 charCount++;
59             }
60         }
61
62         if(boardWidth == -1){
63             boardWidth = charCount;
64         }
65         else if(charCount != boardWidth){
66             std::cerr << "error: ukuran papan tidak
67                 konsisten" << std::endl;
68             isValid = false;
69             return 1;
70         }
71
72         //simpan koordinat karakter non spasi
73         int colIdx = 0;
74         for(int x = 0; x < static_cast<int>(line.size());
75             x++){
76             char c = line[x];
77             if(c != ' ' && c != '\t'){
78                 colors_coordinate[c].push_back({colIdx, y
79                     });
80                 colIdx++;
81             }

```

```

77     }
78     ++y;
79 }
80
81 file.close();
82 //=====VALIDASI INPUT
83     =====
84 if(board.empty()){
85     std::cerr << "error: papan tidak boleh kosong!"
86     << std::endl;
87     return 1;
88 }
89
90 if(board.size() != static_cast<size_t>(boardWidth)){
91     std::cerr << "error: papan harus persegi" << std
92     ::endl;
93     return 1;
94 }
95
96 if(colors_coordinate.size() != board.size()){
97     std::cerr << "error: jumlah warna harus sama
98     dengan ukuran papan" << std::endl;
99     return 1;
100 }
101
102 std::cout << "input valid" << std::endl;
103 std::cout << "jumlah warna: " << colors_coordinate.
104 size() << std::endl;
105
106 //=====ALGORITMA INTI
107     =====
108
109 //taruh queens di koordinat awal dari setiap warna
110 for(const auto& pair : colors_coordinate){
111     char color = pair.first;
112     const std::vector<coordinates>& coords = pair.
113     second;
114     if(!coords.empty()){
115         queens_position[color] = coords[0];
116     }

```

```

110     }
111
112     ShiftQueens(queens_position);
113
114 }

```

## 4.2 queens.cpp

Listing 3: Implementasi Algoritma Brute Force

```

1  #include "queens.h"
2
3  std::map<char, std::vector<coordinates>>
4      colors_coordinate;
5  std::map<char, coordinates> queens_position;
6  std::vector<std::string> board;
7
8  void clearScreen(){
9      system("cls");
10 }
11
12 void ShiftQueens(std::map<char, coordinates>&
13     queens_position){
14     //array of char untuk menyimpan warna-warna yang ada
15     std::vector<char> colors;
16     int n, carry, iterationCount = 0;
17     //array of int untuk menyimpan index koordinat yang
18     //dipilih untuk setiap warna
19     std::vector<int> indices;
20     std::vector<int> maxIndices;
21     coordinates posA, posB;
22     std::map<char, coordinates> tempPos;
23     bool found, allValid;
24     float waktuMs;
25
26     //buat daftar warna dari colors_coordinate
27     for(const auto& pair : colors_coordinate){
28         colors.push_back(pair.first);
29     }
30     n = colors.size();

```

```

28
29 //inisialisasi indices dengan 0 untuk semua warna
30 indices.resize(n, 0);
31
32 //hitung jumlah koordinat max setiap warna
33 for(int i = 0; i < n; i++){
34     maxIndices.push_back(colors_coordinate[colors[i]
35         ].size());
36 }
37 found = false;
38
39 //algoritma brute force: coba semua kombinasi
40 auto startTime = std::chrono::high_resolution_clock::
41     now();
42 while(!found){
43     iterationCount++;
44
45     for(int i = 0; i < n; i++){
46         tempPos[colors[i]] = colors_coordinate[colors
47             [i]][indices[i]];
48     }
49
50     clearScreen();
51     std::cout << "konfigurasi ke-" << iterationCount
52         << ":" << std::endl;
53     printBoard(tempPos);
54     std::this_thread::sleep_for(std::chrono::
55         milliseconds(300)); //tunggu 0.3 detik
56
57     //cek apakah ada konflik antara queen
58     allValid = true;
59     for(int i = 0; i < n && allValid; i++){
60         for(int j = i + 1; j < n && allValid; j++){
61             posA = tempPos[colors[i]];
62             posB = tempPos[colors[j]];
63
64             //cek baris, kolom, diagonal bersentuhan
65             (jarak 1)
66             if(posA.x == posB.x || posA.y == posB.y
67                 ||

```

```

61         (abs(posA.x - posB.x) == 1 && abs(posA
62             .y - posB.y) == 1)){
63             allValid = false;
64         }
65     }
66
67     if(allValid){
68         queens_position = tempPos;
69         found = true;
70     }
71     else{
72         //increment indices
73         carry = n - 1;
74         while(carry >= 0){
75             indices[carry]++;
76             if(indices[carry] < maxIndices[carry]){
77                 break;
78             }
79             indices[carry] = 0; //reset dan carry ke
80                 digit berikutnya
81             carry--;
82         }
83
84         if(carry < 0){
85             //sudah mencoba semua kombinasi
86             std::cout << "Tidak ada solusi" << std::
87                 endl;
88             break;
89         }
90     }
91
92     auto endTime = std::chrono::high_resolution_clock::
93         now();
94     waktuMs = std::chrono::duration<float, std::milli>(
95         endTime - startTime).count();
96
97     std::cout << "hasil akhir: " << std::endl;
98     printBoard(queens_position);

```

```

96
97     std::cout << "jumlah konfigurasi yg telah ditinjau: "
          << iterationCount << std::endl;
98     std::cout << "waktu eksekusi: " << waktuMs << " ms"
          << std::endl;
99
100 }
101
102 void printBoard(const std::map<char, coordinates>&
103               queens_pos){
104     //buat salinan board untuk dimodifikasi
105     std::vector<std::string> output = board;
106
107     //tandai posisi queen dengan '#'
108     for(const auto& pair : queens_pos){
109         coordinates pos = pair.second;
110         if(pos.y >= 0 && pos.y < static_cast<int>(output.
111             size())){
112             int colIdx = 0;
113             for(int i = 0; i < static_cast<int>(output[
114                 pos.y].size()); i++){
115                 if(output[pos.y][i] != ' ' && output[pos.
116                     y][i] != '\t'){
117                     if(colIdx == pos.x){
118                         output[pos.y][i] = '#';
119                         break;
120                     }
121                     colIdx++;
122                 }
123             }
124         }
125     }
126
127     //cetak board
128     for(const auto& line : output){
129         std::cout << line << std::endl;
130     }
131 }

```

## 4.3 queens.h

Listing 4: Implementasi queens.h

```
1  #include <iostream>
2  #include <map>
3  #include <utility>
4  #include <vector>
5  #include <chrono>
6  #include <thread>
7
8  struct coordinates{
9      int x;
10     int y;
11 };
12
13 extern std::map<char, std::vector<coordinates>>
14     colors_coordinate;
15 extern std::map<char, coordinates> queens_position;
16 extern std::vector<std::string> board;
17
18 void ShiftQueens(std::map<char, coordinates>&
19     queens_position);
20 void printBoard(const std::map<char, coordinates>&
21     queens_pos);
```

## 4.4 index.html

Listing 5: Implementasi index.html

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Queens Solver</title>
6      <style>
7          body { font-family: sans-serif; background:
8              #985555; color: #eee; display: flex; flex-
9              direction: column; align-items: center;
10             padding: 30px; }
11          h2 { margin-bottom: 16px; }
```



```

9         input[type="file"] { display: none; }
10         button { background: #3f292d; color: #fff; border
           : none; padding: 8px 20px; border-radius: 6px;
           cursor: pointer; margin: 6px; font-size: 20px
           ; }
11         button:disabled { opacity: .4; cursor: default; }
12         #board { display: grid; gap: 2px; margin: 16px 0;
           }
13         .cell { width: 44px; height: 44px; display: flex;
           align-items: center; justify-content: center;
           border-radius: 4px; font-size: 24px; }
14         .info { font-size: 13px; color: #aaa; margin: 4px
           0; }
15         canvas { display: none; }
16     </style>
17 </head>
18 <body>
19     <h2>Queens Solver</h2>
20     <div>
21         <button onclick="document.getElementById('f').
           click()">Input .txt</button>
22         <button id="solveBtn" onclick="solve()" disabled>
           Solve</button>
23         <button id="saveBtn" onclick="saveImage()"
           disabled>Save as Image</button>
24         <input type="file" id="f" accept=".txt" onchange
           ="load(event)">
25     </div>
26     <div class="info" id="status"></div>
27     <div id="board"></div>
28     <div class="info" id="stats"></div>
29     <canvas id="canvas"></canvas>
30
31 <script>
32 const COLORS = ['#FF0000', '#00CC00', '#0066FF', '#FFD700
           ', '#FF00FF', '#FF8800', '#00CCCC', '#8800CC', '#CCCC00', '#
           CC0066', '#0000CC', '#00FF88', '#CC4400', '#8888FF
           ', '#006644', '#FF6699', '#444444', '#88DDFF', '#AA5500
           ', '#00AA00', '#DD0088', '#5555FF', '#BBBB00', '#FF4444
           ', '#009999', '#CC88FF'];

```

```

33     let bd=[],sz=0,cm={},cc={};
34
35     function load(e){
36         const r=new FileReader();
37         r.onload=function(ev){
38             const lines=ev.target.result.split('\n').map(l=>l
39                 .replace(/\r/,')).trimEnd()).filter(l=>l);
40             bd=[];cc={};cm={};let ci=0;
41             for(let y=0;y<lines.length;y++){
42                 const row=[];let col=0;
43                 for(let x=0;x<lines[y].length;x++){
44                     const c=lines[y][x];
45                     if(c!=' ' && c!='\t'){
46                         row.push(c);
47                         if(!cc[c]){cc[c]=[];cm[c]=COLORS[ci
48                             ++%COLORS.length];}
49                         cc[c].push({x:col,y});col++;
50                     }
51                 }
52                 bd.push(row);
53             }
54             sz=bd.length;
55             if(Object.keys(cc).length!==sz){status('Error:
56                 warna ≠ ukuran papan',1);return;}
57             render({});status('File loaded. '+sz+'x'+sz+', '+
58                 Object.keys(cc).length+' warna');
59             document.getElementById('solveBtn').disabled=
60                 false;
61         };
62         r.readAsText(e.target.files[0]);
63     }
64
65     function render(qp){
66         const b=document.getElementById('board');
67         b.style.gridTemplateColumns='repeat(${sz},44px)';b.
68             innerHTML='';
69         for(let y=0;y<sz;y++)for(let x=0;x<sz;x++){
70             const d=document.createElement('div');d.className
71                 ='cell';
72             d.style.background=cm[bd[y][x]]||'#333';

```

```

66         for(const p of Object.values(qp))if(p.x===x&&p.y
67             ===y)d.textContent='Q';
68     b.appendChild(d);
69 }
70
71 function status(m,err){document.getElementById('status').
    style.color=err?'#e94560':'#aaa';document.
    getElementById('status').textContent=m;}
72
73 async function solve(){
74     const colors=Object.keys(cc),n=colors.length,idx=new
        Array(n).fill(0),mx=colors.map(c=>cc[c].length);
75     let it=0;const t0=performance.now();
76     document.getElementById('solveBtn').disabled=true;
77     while(true){
78         it++;
79         const tp={};for(let i=0;i<n;i++)tp[colors[i]]=cc[
            colors[i]][idx[i]];
80         if(it%500===0){render(tp);status('Iterasi: '+it);
            await new Promise(r=>setTimeout(r,0));}
81         let ok=true;
82         for(let i=0;i<n&&ok;i++)for(let j=i+1;j<n&&ok;j
            ++){
83             const a=tp[colors[i]],b=tp[colors[j]];
84             if(a.x===b.x||a.y===b.y||(Math.abs(a.x-b.x)
                ===1&&Math.abs(a.y-b.y)===1))ok=false;
85         }
86         if(ok){
87             render(tp);
88             const ms=(performance.now()-t0).toFixed(1);
89             status('Solusi ditemukan!');
90             document.getElementById('stats').textContent=
                it+' iterasi, '+ms+' ms';
91             document.getElementById('saveBtn').disabled=
                false;
92             document.getElementById('solveBtn').disabled=
                false;
93             return;
94         }

```

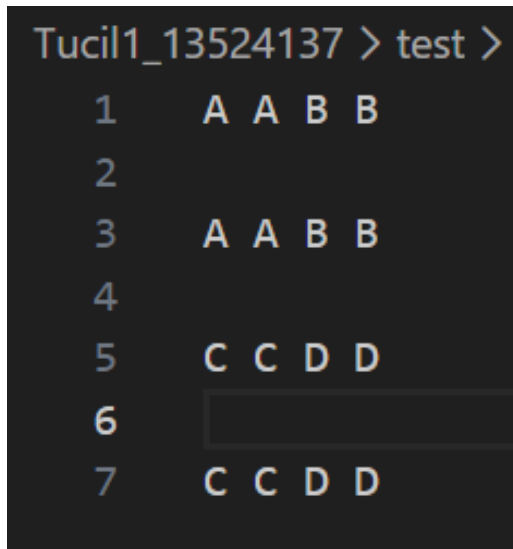
```

95         let c=n-1;
96         while(c>=0){idx[c]++;if(idx[c]<mx[c])break;idx[c
97             ]=0;c--;}
98         if(c<0){status('Tidak ada solusi',1);document.
99             getElementById('solveBtn').disabled=false;
100             return;}
101     }
102 }
103
104 function saveImage(){
105     const s=50,gap=2,total=sz*s+(sz-1)*gap;
106     const cv=document.getElementById('canvas');cv.width=
107         total;cv.height=total;
108     const ctx=cv.getContext('2d');
109     ctx.fillStyle='#111';ctx.fillRect(0,0,total,total);
110     const cells=document.getElementById('board').children
111         ;
112     for(let y=0;y<sz;y++)for(let x=0;x<sz;x++){
113         const cell=cells[y*sz+x];
114         const px=x*(s+gap),py=y*(s+gap);
115         ctx.fillStyle=cm[bd[y][x]]||'#333';
116         ctx.beginPath();ctx.roundRect(px,py,s,s,4);ctx.
117             fill();
118         if(cell.textContent==='Q'){
119             ctx.fillStyle='#fff';ctx.font='28px sans-
120                 serif';ctx.textAlign='center';ctx.
121                 textBaseline='middle';
122             ctx.fillText('Q',px+s/2,py+s/2);
123         }
124     }
125     const a=document.createElement('a');
126     a.download='queens_solution.png';a.href=cv.toDataURL
127         ('image/png');a.click();
128 }
129 </script>
130 </body>
131 </html>

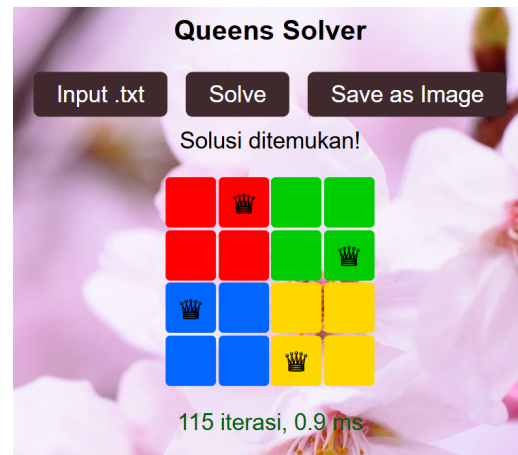
```

## 5 Hasil Pengujian (Tangkapan Layar)

Berikut adalah 5 contoh input dan output dari program "Queens Solver" yang telah dijalankan:

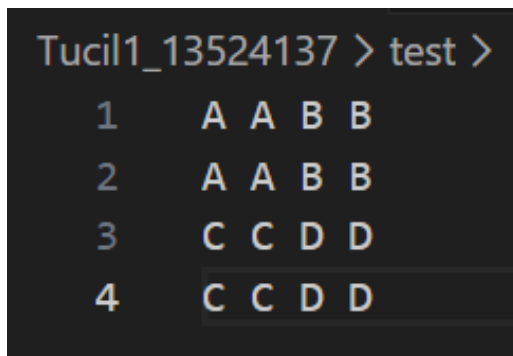


(a) Input: Grid 4x4



(b) Output: Solusi Ditemukan

Gambar 1: Kasus Grid 4x4 dengan extra spasi

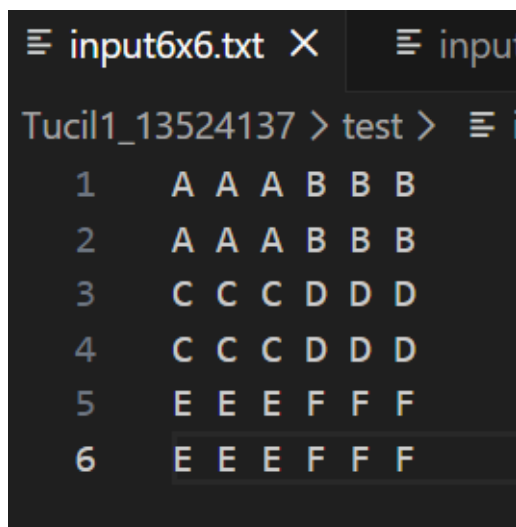


(a) Input: Grid 4x4

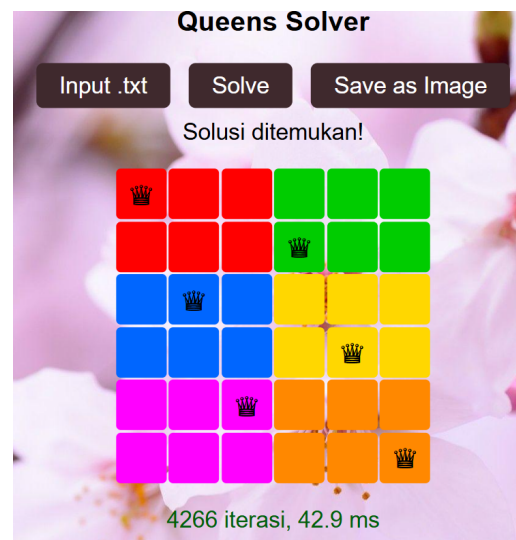


(b) Output: Solusi Ditemukan

Gambar 2: Kasus Grid 4x4

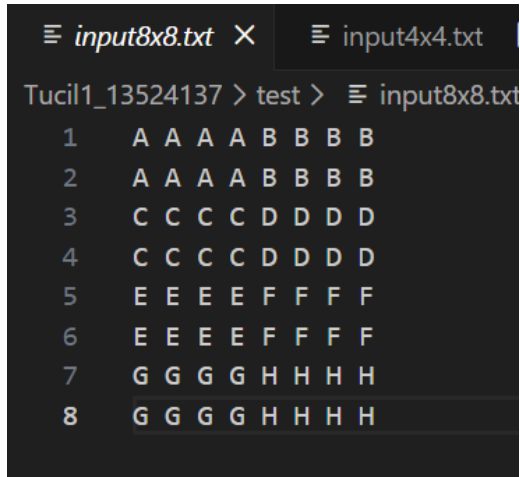


(a) Input: Grid 6x6

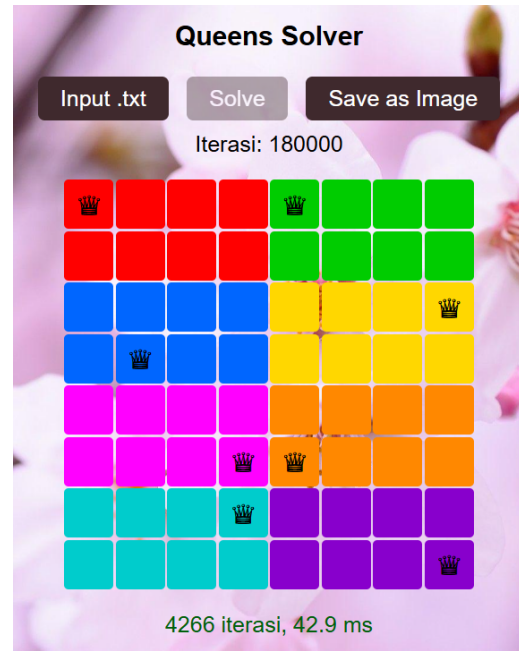


(b) Output: Solusi Ditemukan

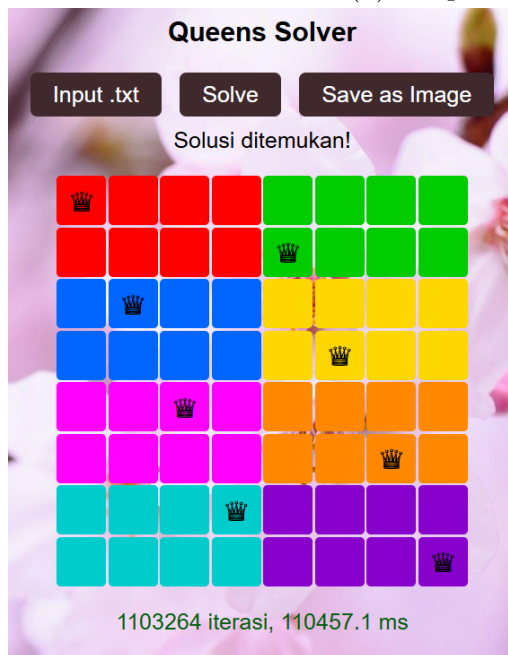
Gambar 3: Kasus Grid 6x6



(a) Input: Grid 8x8

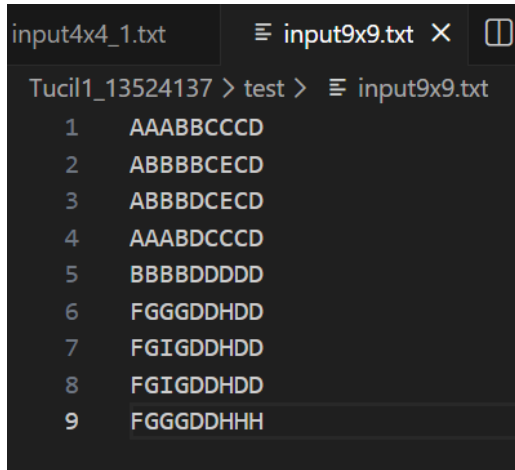


(b) Output: proses iterasi ke-sekian

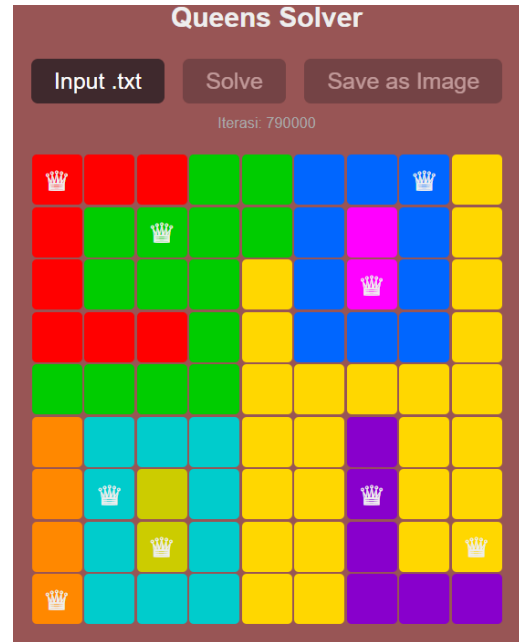


(c) Output: Solusi Ditemukan

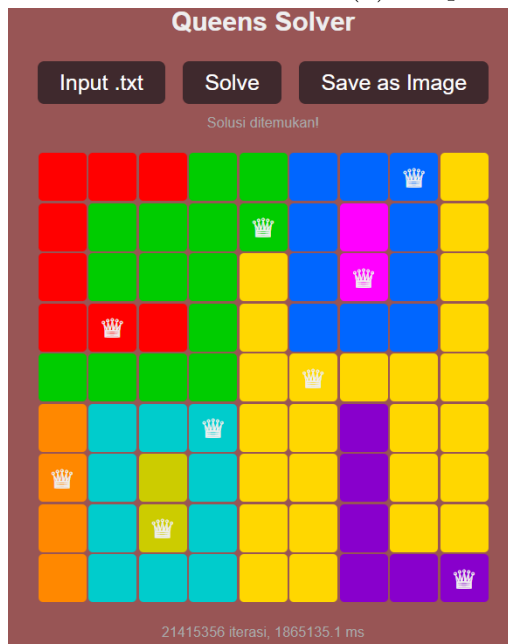
Gambar 4: Kasus Grid 8x8



(a) Input: Grid 9x9



(b) Output: proses iterasi ke-sekian



(c) Output: Solusi Ditemukan

Gambar 5: Kasus Grid 9x9

notes: logika pemrogramannya sama, hanya layout GUI-nya saja yang sebelum dipercantik

## 6 Pranala Repositori

Seluruh kode program dan dokumen pendukung dapat diakses melalui tautan berikut:



[https://github.com/kurakuraninja00/Tucil1\\_13524137](https://github.com/kurakuraninja00/Tucil1_13524137)

## Pernyataan Orisinalitas

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

A handwritten signature in black ink, appearing to read 'Reysha', with a horizontal line underneath.

Reysha Syafitri MR  
NIM: 13524137

## A Lampiran

Berikut adalah tabel evaluasi pemenuhan spesifikasi program:

| No | Poin                                                              | Ya | Tidak |
|----|-------------------------------------------------------------------|----|-------|
| 1  | Program berhasil di kompilasi tanpa kesalahan                     | ✓  |       |
| 2  | Program berhasil di jalankan                                      | ✓  |       |
| 3  | Solusi yang diberikan program benar dan mematuhi aturan permainan | ✓  |       |
| 4  | Program dapat membaca masukan berkas .txt                         | ✓  |       |
| 5  | Program memiliki Graphical User Interface (GUI)                   | ✓  |       |
| 6  | Program dapat menyimpan solusi dalam bentuk file gambar           | ✓  |       |

Tabel 1: Checklist Evaluasi Program